

Vite & Gourmand — Site Traiteur Bordeaux

Projet ECF — TP Développeur Web et Web Mobile (Studi)

Antoine Banzet · 2026

Présentation

Site vitrine et de commande pour **Vite & Gourmand**, traiteur à Bordeaux depuis 1999. Permet aux visiteurs de consulter les menus, aux clients de passer commande et de déposer un avis. Inclut un espace employé (gestion des menus / plats / horaires / commandes / avis) et un espace administration (gestion des comptes employés, statistiques).

 Choix techniques et justifications détaillées : voir le **dossier technique** (document séparé).

Stack technique

Couche	Technologie
Frontend	HTML5 · CSS3 (custom properties) · JavaScript vanilla
Backend	PHP 8.1 · PDO · API REST JSON (sans framework)
Base SQL	MySQL 8+ · InnoDB · utf8mb4
Base NoSQL	MongoDB (statistiques admin uniquement)
Authentification	Sessions PHP (cookie <code>HttpOnly</code>) · bcrypt cost 12
Emails	<code>mail()</code> natif en local · API Brevo en production

Structure du projet

```
projet/
├── frontend/                               Pages HTML + CSS + JS
│   ├── index.html                         Accueil
│   ├── menu.html                         Catalogue des menus
│   ├── commande.html                     Tunnel de commande
│   ├── panier.html                       Liste de sélection de menus
│   ├── contact.html                     Contact / devis
│   ├── MonCompte.html                   Espace client
│   ├── SeConnecter.html                 Connexion
│   ├── SInscrire.html                   Inscription
│   ├── reset-password.html              Réinitialisation / définition du mot de passe
│   ├── employe.html                     Connexion espace employé
│   ├── employe-dashboard.html            Tableau de bord employé
│   ├── admin.html                       Connexion administration
│   ├── admin-dashboard.html              Tableau de bord admin
│   ├── cgv.html · rgpd.html · legal.html · accessibilite.html  Pages légales
│   ├── style.css                         Feuille de styles unique
│   ├── navbar-inject.js                  Génère et injecte le HTML de la navbar (source unique)
│   ├── navbar.js                         Comportement navbar : burger, état connecté, badge panier
│   ├── panier.js                         Utilitaires panier partagés (localStorage)
│   ├── images/                           Assets visuels
│   └── vite_et_gourmand.sql              Schéma SQL + données de test
└── api/                                   Backend PHP
    ├── index.php                         Routeur principal (Front Controller)
    ├── config.php                       Connexion BDD / mail / Mongo
    ├── helpers.php                      Réponses JSON, authentification, validations
    ├── mongodb.php                      Connexion MongoDB + dégradation gracieuse
    ├── .env.example                     Variables d'environnement (modèle)
    ├── aiven-ca.pem                     Certificat SSL MySQL (production Aiven)
    ├── Dockerfile · docker-entrypoint.sh  Image de déploiement
    └── controllers/
        ├── auth.php · menus.php · plats.php · commandes.php
        ├── avis.php · utilisateurs.php · admin.php
        └── themes.php · regimes.php · allergenes.php · horaires.php · contact.php

utile/                                     Livrables de conception (hors dépôt)
├── diagramme_classes.png
├── diagramme_utilisation.png
├── diagramme_sequence_connexion.png
├── diagramme_sequence_commande.png
├── charte_graphique_complete.pdf
└── Wireframes vite et gourmand/
```

Installation locale

Prérequis

- PHP 8.1+
- MySQL 8+
- Serveur web avec `mod_rewrite` activé (Apache/Nginx), ou XAMPP / WAMP / Laragon
- Extension PHP `mongodb` (optionnelle — sans elle, seul le graphique admin est désactivé)

1. Base de données

Le fichier `vite_et_gourmand.sql` crée lui-même la base (`DROP` puis `CREATE DATABASE`) : il faut donc l'importer **sans** préciser de base de données.

```
mysql --default-character-set=utf8mb4 -u root -p < projet/frontend/vite_et_gourmand.sql
```

Le paramètre `--default-character-set=utf8mb4` est important : sans lui, l'import peut corrompre les caractères accentués (ex : « Entrée » devient « EntrÃ©e »).

2. Configuration backend

```
cp projet/api/.env.example projet/api/.env
```

```
DB_HOST=localhost
DB_PORT=3306
DB_NAME=vite_et_gourmand
DB_USER=root
DB_PASS=votre_mot_de_passe

MAIL_FROM=antoinebaban@gmail.com
MAIL_NAME=Vite & Gourmand
APP_URL=http://localhost/ViteEtGourmand/projet/frontend

# MongoDB – laisser vide pour désactiver le graphique admin (l'app fonctionne sans)
MONGO_URI=
MONGO_DB=vite_et_gourmand_logs

# Brevo – laisser vide en local : le code utilise mail() nativement
BREVO_API_KEY=
```

En production, `MAIL_FROM` doit être une adresse vérifiée comme expéditeur dans le compte Brevo (Expéditeurs, domaines et IP dédiée). Le domaine `viteetgourmand.fr` étant fictif, une adresse personnelle vérifiée est utilisée à sa place — le nom affiché reste « Vite & Gourmand » via `MAIL_NAME` . Sans expéditeur vérifié, Brevo rejette l'envoi silencieusement (aucune erreur remontée côté application).

3. Initialisation des mots de passe

Les mots de passe importés par vite_et_gourmand.sql sont des marqueurs (\$2y\$12\$PLACEHOLDER_RUN_SETUP), pas de vrais hachages — il faut les générer une fois localement.

Pour chaque mot de passe de test, générer son hachage bcrypt :

```
php -r "echo password_hash('Admin2026!', PASSWORD_BCRYPT, ['cost'=>12]);"
```

Puis, dans phpMyAdmin (ou mysql), coller le hachage obtenu :

```
UPDATE utilisateur SET password = '<hash copié>' WHERE email = 'admin@viteetgourmand.fr';
UPDATE utilisateur SET password = '<hash copié>' WHERE email = 'employe@viteetgourmand.fr';
UPDATE utilisateur SET password = '<hash copié>' WHERE email IN
('jean.martin@email.fr', 'sophie.b@email.fr', 'pierre.d@email.fr');
```

(Les trois comptes clients partagent le même mot de passe Client2026! , donc le même hachage.)

4. Lancer le projet

Ouvrir http://localhost/ViteEtGourmand/projet/frontend/index.html

Comptes de test

Rôle	Email	Mot de passe
Administrateur	admin@viteetgourmand.fr	Admin2026!
Employé	employe@viteetgourmand.fr	Employe2026!
Client 1	jean.martin@email.fr	Client2026!
Client 2	sophie.b@email.fr	Client2026!
Client 3	pierre.d@email.fr	Client2026!

Endpoints API principaux

POST	/api/auth/login	
POST	/api/auth/register	
POST	/api/auth/logout	
GET	/api/auth/me	
POST	/api/auth/forgot-password	
POST	/api/auth/reset-password	
GET	/api/menus	filtres : theme_id, regime_id, prix_min, prix_max, personnes, q
GET	/api/menus/populaires	
GET	/api/menus/{id}	
POST	/api/commandes	
GET	/api/commandes/mes-commandes	
GET	/api/commandes	employé / admin
PUT	/api/commandes/{id}	
PUT	/api/commandes/{id}/statut	employé / admin
PUT	/api/commandes/{id}/annuler	
GET	/api/commandes/{id}/historique	
GET	/api/avis	
POST	/api/avis	
PUT	/api/avis/{id}/validation	employé / admin
GET	/api/admin/stats	
GET	/api/admin/utilisateurs	
POST	/api/admin/employes	
PUT	/api/admin/employes/{id}/desactiver	admin
PUT	/api/admin/employes/{id}/activer	admin
POST	/api/contact	
POST	/api/devis	
POST	/api/factures	demande de facture (client)

Déploiement en production

Architecture : **Render** (PHP via Docker) + **Aiven** (MySQL) + **MongoDB Atlas** + **Brevo** (emails). Démarche complète et justifications : voir le dossier technique, section Déploiement.

- Application en ligne : <https://ecf-dwwwm.onrender.com>
- Dépôt GitHub : <https://github.com/antoinebaban-sketch/ECF-DWWW>

Organisation Git

Branche	Usage
main	Code stable / production
develop	Intégration des fonctionnalités

Projet réalisé en solo : les commits sont faits directement sur `develop` avec des messages atomiques et descriptifs, puis fusionnés dans `main` pour les versions stables.